Лабораторная работа 2
Dockerfile и кэш сборки

Часть 1: разделы 1-5. Материал 03.

Студент: Есімханова Аружан
Группа: 25-ИСТК-2о
Дата: 22 октября 2026 года

Цель работы

Упаковать DemoApi на ASP.NET Core / .NET 10 в Linux-образ с двумя этапами сборки, исключить лишние файлы и подтвердить повторное использование слоя NuGet после изменения исходника.

Среда	Фактическое значение
Docker Client / Server	29.8.0 / 29.8.0
Движок	Docker Desktop, Linux x86_64, WSL 2 включен
Ресурсы Docker	12 CPU, 7.62 ГиБ памяти (требуется не менее 4 ГБ)
PowerShell	7.6.5
Базовые образы	dotnet/sdk:10.0 и dotnet/aspnet:10.0

Комплект сдачи

Dockerfile и .dockerignore приведены полностью; вывод второй сборки содержит CACHED на restore. Запуск hello-world, проверка Dockerfile, реальный контейнер и API проверены. Остался обязательный скриншот Settings → General с флажком WSL 2: автоматический захват окна вернул тайм-аут. Конфигурация подтверждает включенный WSL 2, но скриншот не подменяется текстом.

Публичный репозиторий и все исходники

1. Dockerfile: два этапа

Листинг 1. Полный Dockerfile из корня репозитория. Сначала зависимости, затем исходники; в итоговый образ копируется результат publish.

```
# syntax=docker/dockerfile:1

FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY src/DemoApi/DemoApi.csproj src/DemoApi/
RUN dotnet restore src/DemoApi/DemoApi.csproj
COPY src/ src/
RUN dotnet publish src/DemoApi/DemoApi.csproj -c Release -o /app --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY --from=build /app .
ARG APP_VERSION=dev
ENV APP_VERSION=$APP_VERSION
USER $APP_UID
EXPOSE 8080
HEALTHCHECK --interval=10s --timeout=3s --start-period=15s --retries=3 \
    CMD curl -fsS http://localhost:8080/health/live || exit 1
ENTRYPOINT ["dotnet", "DemoApi.dll"]
```

Этап build использует SDK и выполняет restore/publish в Release. Этап runtime содержит ASP.NET runtime, опубликованное приложение и curl для проверки здоровья. Установка curl и очистка apt объединены в один слой.

USER $APP_UID запускает сервис от пользователя app. EXPOSE документирует внутренний порт 8080. HEALTHCHECK вызывает /health/live. ARG задает публичную версию сборки, ENV делает ее доступной GET /. ENTRYPOINT запускает dotnet как главный процесс.

Dockerfile в GitHub

2. Исключения и базовый образ

Листинг 2. Полный .dockerignore. Дополнительно к образцу методички исключены отчеты, доказательства, документация, Postman и вложенные файлы окружения/сертификатов.

```
**/bin/
**/obj/
**/TestResults/
.git
.github
.vs
.vscode
.idea
tests/
monitoring/
scripts/
db/
docs/
reports/
evidence/
postman/
*.md
Dockerfile
compose*.yaml
compose*.yml
.env
.env.*
**/.env
**/.env.*
**/*.pfx
**/*.p12
```

bin/obj содержат локальные артефакты Windows, которые не должны попадать в Linux-сборку. Git, IDE, тесты и отчеты не нужны для публикации приложения. .env и сертификаты исключаются из контекста, чтобы не оказаться даже во временных слоях.

Проверки подготовки: docker version показал Client и Server; hello-world успешно запустился. Базовый runtime скачан отдельно, его слои и размеры записаны командами image history и image ls. Docker build --check завершился без предупреждений.

Полные журналы подготовки и проверки

3. Сборка и повторное использование кэша

Сначала выполнена сборка 1.0. Затем в TaskService.cs изменено только сообщение создания задачи; файл проекта и список пакетов не менялись. Вторая сборка получила тег 1.1 и APP_VERSION=lab02-1.1.

```
docker build --progress=plain -t devops-labs-demo-api:lab02-1.0 .
docker build --progress=plain --build-arg APP_VERSION=lab02-1.1 `
    -t devops-labs-demo-api:lab02-1.1 .
```

Изменение сообщения (осталось в итоговом коде):

```
Создана задача {TaskId} в проекте {ProjectId}
Создана новая задача {TaskId} в проекте {ProjectId}
```

Листинг 3. Фактический фрагмент второго docker build:

```
#12 [build 3/6] COPY src/DemoApi/DemoApi.csproj src/DemoApi/
#12 CACHED
#13 [build 4/6] RUN dotnet restore src/DemoApi/DemoApi.csproj
#13 CACHED
#14 [build 5/6] COPY src/ src/
#14 DONE 0.0s
#15 [build 6/6] RUN dotnet publish src/DemoApi/DemoApi.csproj -c Release -o /app --no-restore
#15 3.234   DemoApi -> /src/src/DemoApi/bin/Release/net10.0/DemoApi.dll
#15 3.289   DemoApi -> /app/
#15 DONE 3.4s
```

Файл проекта и restore взяты из кэша; копирование исходников и publish выполнены заново. Следовательно, изменение кода не потребовало повторной загрузки NuGet-пакетов. SHA-256 до/после изменения и результаты проверки шагов сохранены в cache-check.json.

Артефакт	Фактическое значение
Итоговый образ	devops-labs-demo-api:lab02-1.1
Размер образа (image inspect)	354,253,942 байт (337.8 МиБ)
Платформа / пользователь	linux/amd64, UID 1654

4. Проверка собранного приложения

Эти проверки дополняют обязательную часть 1-5. API запущен в настоящем Linux-контейнере на localhost:8089 → 8080. Другие сервисы и контейнеры не используются для проверки.

Проверка	Результат
Здоровье контейнера	healthy; /health/live выполняется из контейнера
Версия приложения	GET / → 200, version=lab02-1.1
Пользователь процесса	uid=1654(app) gid=1654(app) groups=1654(app)
SDK в итоговом образе	dotnet --list-sdks: пустой вывод
Исходники и тесты в образе	В /app нет C#-исходников, csproj и решения
Live и ready	Оба маршрута → 200
Swagger в Production	GET /swagger/index.html → 404
Перезапуск InMemory	Тот же контейнер: 1 проект до stop/start, 0 после
Swagger без пересборки	Тот же образ в Development: /swagger/index.html → 200
OpenAPI в Development	GET /openapi/v1.json → 200, stats присутствует
Учебные сбои отключены	Chaos__Enabled=false, /api/chaos/error → 404
HTTP-сценарии API	26 запросов: 23 сценария и 3 удаления тестовых проектов
Коллекция Postman CLI	24 запроса, 73 проверки, 0 ошибок; exit code 0
Регрессия .NET Release	28/28 тестов, 0 failed, 0 skipped; InMemory

Ответ GET /:

```
{
  "service": "DemoApi",
  "version": "lab02-1.1",
  "host": "e7ee83006c4d"
}
```

Полные запросы/ответы, журналы контейнера, статус здоровья и результаты Postman лежат в evidence/lab02. API проверялся локально; результаты в облако Postman не публиковались.

5. Ответы на вопросы и итог

Почему контейнер стартует быстрее VM?

Не требуется загружать отдельное ядро и полноценную ОС. В WSL 2 уже работает ядро Linux; Docker создает namespaces/cgroups, монтирует слои и запускает процесс приложения.

Почему сначала копируется csproj, затем код?

Список зависимостей меняется реже C#-файлов. Отдельный слой restore остается в кэше после изменения кода. Это подтверждено второй сборкой.

Что попадает в итоговый образ после multi-stage?

ASP.NET runtime, опубликованные DLL и зависимости, конфигурация, curl. SDK, исходники и промежуточные obj остаются на этапе build. Отсутствие SDK проверено через dotnet --list-sdks.

Почему в контейнере нет Swagger? Как включить?

По умолчанию окружение Production; Program.cs регистрирует Swagger только в Development. Новый контейнер из того же образа с ASPNETCORE_ENVIRONMENT=Development открыл Swagger без пересборки. Chaos__Enabled=false отключает учебные сбои.

Что будет с проектами после docker stop/start?

InMemory живет в оперативной памяти процесса. После его завершения проекты теряются, даже если контейнер тот же. Фактическая проверка создала проект, остановила/запустила контейнер и получила пустой список.

Итог и ограничение

Техническая часть материала 03 выполнена и подтверждена сохраненными запусками. Для полного оформления контрольной точки требуется добавить настоящий скриншот Docker Desktop Settings → General с включенным WSL 2. Отдельная часть 2 (материал 04) в исходном архиве отсутствует и не объявляется выполненной.

Источники и материалы

Условия: материал 03, «Неделя 3 - Контрольная точка 2», лабораторная 2, часть 1, и лекция 2 из загруженного архива Devops (1).zip.

Docker: правила кэша · Microsoft: образы ASP.NET Core

Инструкции и ответы в репозитории · Сохраненные доказательства